Training a neural network on MNIST with Keras

This simple example demonstrates how to plug TensorFlow Datasets (TFDS) into a Keras model.


```
import tensorflow as tf
import tensorflow_datasets as tfds
```

```
2023-10-03 09:29:30.258272: E tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:9342] Unable to r
2023-10-03 09:29:30.258321: E tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.cc:609] Unable to re
2023-10-03 09:29:30.258358: E tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:1518] Unable to
```

Step 1: Create your input pipeline

Start by building an efficient input pipeline using advices from:

- The Performance tips (https://www.tensorflow.org/datasets/performances) guide
- The Better performance with the `tf.data` API (https://www.tensorflow.org/guide/data_performance#optimize_performance) guide

Load a dataset

Load the MNIST dataset with the following arguments:

- `shuffle_files=True`: The MNIST data is only stored in a single file, but for larger datasets with multiple files on disk, it's good practice to shuffle them when training.
- `as_supervised=True`: Returns a tuple `(img, label)` instead of a dictionary `{'image': img, 'label': label}`.

```
(ds_train, ds_test), ds_info = tfds.load(
    'mnist',
    split=['train', 'test'],
    shuffle_files=True,
    as_supervised=True,
    with_info=True,
)
```

```
2023-10-03 09:29:33.682941: E tensorflow/compiler/xla/stream_executor/cuda/cuda_driver.cc:268] failed ca
```

Build a training pipeline

Apply the following transformations:

- `tf.data.Dataset.map` (https://www.tensorflow.org/api_docs/python/tf/data/Dataset#map): TFDS provide images of type `tf.uint8` (https://www.tensorflow.org/api_docs/python/tf#uint8), while the model expects `tf.float32` (https://www.tensorflow.org/api_docs/python/tf#float32). Therefore, you need to normalize images.
- `tf.data.Dataset.cache` (https://www.tensorflow.org/api_docs/python/tf/data/Dataset#cache) As you fit the dataset in memory, cache it before shuffling for a better performance.
  Note: Random transformations should be applied after caching.
- `tf.data.Dataset.shuffle` (https://www.tensorflow.org/api_docs/python/tf/data/Dataset#shuffle): For true randomness, set the shuffle buffer to the full dataset size.
  Note: For large datasets that can't fit in memory, use `buffer_size=1000` if your system allows it.
- `tf.data.Dataset.batch` (https://www.tensorflow.org/api_docs/python/tf/data/Dataset#batch): Batch elements of the dataset after shuffling to get unique batches at each epoch.
- `tf.data.Dataset.prefetch` (https://www.tensorflow.org/api_docs/python/tf/data/Dataset#prefetch): It is good practice to end the pipeline by prefetching for performance (https://www.tensorflow.org/guide/data_performance#prefetching).

```
def normalize_img(image, label):
  """Normalizes images: `uint8` -> `float32`."""
  return tf.cast(image, tf.float32) / 255., label

ds_train = ds_train.map(
    normalize_img, num_parallel_calls=tf.data.AUTOTUNE)
ds_train = ds_train.cache()
ds_train = ds_train.shuffle(ds_info.splits['train'].num_examples)
ds_train = ds_train.batch(128)
ds_train = ds_train.prefetch(tf.data.AUTOTUNE)
```

Build an evaluation pipeline

Your testing pipeline is similar to the training pipeline with small differences:

- You don't need to call `tf.data.Dataset.shuffle` (https://www.tensorflow.org/api_docs/python/tf/data/Dataset#shuffle).
- Caching is done after batching because batches can be the same between epochs.

```
ds_test = ds_test.map(
    normalize_img, num_parallel_calls=tf.data.AUTOTUNE)
ds_test = ds_test.batch(128)
ds_test = ds_test.cache()
ds_test = ds_test.prefetch(tf.data.AUTOTUNE)
```

Step 2: Create and train the model

Plug the TFDS input pipeline into a simple Keras model, compile the model, and train it.

```
model = tf.keras.models.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dense(10)
])
model.compile(
    optimizer=tf.keras.optimizers.Adam(0.001),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[tf.keras.metrics.SparseCategoricalAccuracy()],
)

model.fit(
    ds_train,
    epochs=6,
    validation_data=ds_test,
)
```

```
Epoch 1/6
469/469 [==============================] - 4s 4ms/step - loss: 0.3621 - sparse_categorical_accuracy: 0.
Epoch 2/6
469/469 [==============================] - 1s 3ms/step - loss: 0.1602 - sparse_categorical_accuracy: 0.
Epoch 3/6
469/469 [==============================] - 1s 2ms/step - loss: 0.1174 - sparse_categorical_accuracy: 0.
Epoch 4/6
469/469 [==============================] - 1s 3ms/step - loss: 0.0911 - sparse_categorical_accuracy: 0.
Epoch 5/6
469/469 [==============================] - 1s 2ms/step - loss: 0.0738 - sparse_categorical_accuracy: 0.
Epoch 6/6
469/469 [==============================] - 1s 2ms/step - loss: 0.0617 - sparse_categorical_accuracy: 0.
<keras.src.callbacks.History at 0x7fc41e0cb880>
```


Last updated 2023-10-03 UTC.